МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)
на тему:

«Система автоматизованого аналізу студентських робіт»

Виконав студент групи ІПЗ-21-3
ОСИПЧУК Антон Олексійович

Керівник роботи:
МОРОЗОВ Андрій Васильович

Рецензент:
ЛОБАНЧИКОВА Надія Миколаївна

Житомир – 2025

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»
Зав. кафедри інженерії
програмного забезпечення
Тетяна Вакалюк
«26» лютого 2025 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: **ОСИПЧУК Антон Олексійович**
Керівник роботи: **МОРОЗОВ Андрій Васильович**
Тема роботи: **«Система автоматизованого аналізу студентських робіт»**, затверджена Наказом закладу вищої освіти від **«14» лютого 2025 р., №61/с**
Вихідні дані для роботи: **використання сучасних технологій для проектування, розробки та реалізації системи автоматизованого аналізу студентських робіт**
Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	**Консультант**	Завдання видав	Завдання прийняв
1	Морозов А.В.	27.02.2025р.	27.02.2025р.
2	Морозов А.В.	27.02.2025р.	27.02.2025р.
3	Морозов А.В.	27.02.2025р.	27.02.2025р.

РЕФЕРАТ

Випускна кваліфікаційна робота бакалавра складається з опису системи автоматизованого аналізу студентських письмових робіт та пояснювальної записки. Пояснювальна записка до випускної роботи містить 99 сторінки, 99 ілюстрацій та 9 таблиць.

Метою роботи є розробка інтегрованого рішення, яке забезпечує користувачам зручну платформу для завантаження та перегляду письмових робіт у форматі PDF, автоматичного аналізу їхнього змісту з використанням методів AI, генерування короткого резюме та попереднього виставлення оцінок за заданими критеріями, із подальшою можливістю ручного коригування.

Основна ідея полягає у створенні простого та інтуїтивно зрозумілого інтерфейсу, що підтримує рольову модель користувачів (студент/викладач), дає змогу студентам безпечно завантажувати роботи, а викладачам — переглядати результати аналізу, виставляти оцінки та стежити за успішністю студентів.

У рамках проєкту було визначено ключові завдання розробки, проведено аналіз функціональних вимог до системи, а також розглянуто подібні сервіси, що вже існують на ринку. Було здійснено розробку моделі даних для зберігання студентських робіт, результатів аналізу та історії оцінювання із застосуванням PostgreSQL. Була спроєктована багаторівнева архітектура з інтеграція з AI-модулем на базі Semantic Kernel для виклику LLM-моделі та обробки тексту, а також налаштування Blob Storage для зберігання файлів. Для зручного користування системою було розроблено зручний та технологічний фронтенд на базі бібліотеки React.js.

КЛЮЧОВІ СЛОВА: СИСТЕМА АВТОМАТИЗОВАНОГО АНАЛІЗУ, СТУДЕНТСЬКІ ПИСЬМОВІ РОБОТИ, AI, REACT, ASP.NET CORE, SEMANTIC KERNEL, POSTGRESQL, VECTOR SEARCH.

ABSTRACT

A bachelor's thesis consists of a description of a system for automated analysis of student written work and an explanatory note. The explanatory note to the final thesis contains 99 pages, 99 illustrations, and 9 tables.

The aim of the work is to develop an integrated solution that provides users with a convenient platform for uploading and viewing written works in PDF format, automatically analyzing their content using AI methods, generating a short summary and preliminary grading according to specified criteria, with the subsequent possibility of manual correction.

The main idea is to create a simple and intuitive interface that supports the role model of users (student/teacher), allows students to securely upload work, and allows teachers to view the results of the analysis, assign grades, and monitor student progress.

The project identified key development tasks, analyzed the functional requirements for the system, and reviewed similar services already on the market. We developed a data model for storing student work, analysis results, and assessment history using PostgreSQL. A multi-level architecture was designed with integration with an AI module based on Semantic Kernel to call the LLM model and process text, as well as setting up Blob Storage for storing files. A user-friendly and technologically advanced frontend based on the React.js library has been developed for convenient use of the system.

KEYWORDS: AUTOMATED ANALYSIS SYSTEM, STUDENT WRITING, AI, REACT, ASP.NET CORE, SEMANTIC KERNEL, POSTGRESQL, VECTOR SEARCH.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API – прикладний програмний інтерфейс

ПЗ – Програмне забезчення

БД – база даних

СУБД – система управління базами даних

UI – користувацький інтерфейс (User Interface)

UX – користувацький досвід (User Experience)

AI – штучний інтелект (Artificial Intelligence)

ML – машинне навчання (Machine Learning)

NLP – обробка природної мови (Natural Language Processing)

LLM – велика мовна модель (Large Language Model)

DTO – об’єкт передачі даних (Data Transfer Object)

DI – впровадження залежностей (Dependency Injection)

ВСТУП

Актуальність теми. У сучасній системі вищої освіти дедалі гостріше стоїть питання оптимізації процесу оцінювання письмових робіт студентів. З введенням дистанційних та змішаних форматів навчання навантаження на викладачів зростає, адже кожен курс може містити десятки, а то й сотні студентських робіт, які потребують детальної перевірки, аналізу та надання індивідуального зворотного зв'язку. Традиційні методи ручної перевірки займають значний час і ресурси, при цьому не завжди забезпечують сталість оцінювання: ступінь об'єктивності може змінюватися від настрою викладача, втоми або суб'єктивного сприйняття тексту.

Автоматизація аналізу письмових робіт із застосуванням сучасних технологій штучного інтелекту та обробки природної мови (NLP), що дозволить значно зменшити час, необхідний на первинну перевірку, стандартизувати критерії оцінювання та сконцентрувати увагу викладача на наданні глибшого аналітичного зворотного зв'язку. Водночас така система стає потужним інструментом для студентів: вони можуть отримувати миттєві рекомендації щодо покращення структури викладених думок, стилістики та оформлення, що сприяє підвищенню якості навчальних робіт.

Мета кваліфікаційної роботи – розробка веб-застосуноку "Student Grading System", який забезпечить автоматизований аналіз студентських письмових робіт формату PDF, генерацію резюме, попереднє виставлення оцінок за заданими критеріями та надання ролей "Студент" і "Викладач" з відповідними можливостями взаємодії.

Для реалізації системи автоматизованого аналізу студентських письмових робіт обрано сучасний стек. ASP.NET Core забезпечить надійну та масштабовану бекенд-архітектуру з чіткою розподільною моделлю (сервіси, репозиторії, кешування). React із Tailwind CSS дозволить створити адаптивний, зручний для користувача інтерфейс, який підтримує швидку взаємодію та багатомовність.

Ефективне зберігання текстових даних та векторний пошук семантичних представлень забезпечить PostgreSQL із розширенням pgvector. В свою чергу для аналізу тексту будемо використовувати вбудовану в .Net бібліотеку Semantic Kernel (з викликом мовної моделі через Deepseek) для витягу ключових ідей та генерування коротких резюме робіт.

Поєднання цих технологій дозволяє створити систему з високим рівнем продуктивності, гнучкими можливостями розширення та зручною експлуатацією як викладачами, так і студентами.

Об’єктом дослідження є процес автоматизованого аналізу та оцінювання студентських письмових робіт із використанням методів NLP та прикладних веб-технологій.

Предметом дослідження є методи, алгоритми та інструменти (LLM, векторний пошук, Semantic Kernel, ASP.NET Core, React) для побудови інтегрованої системи автоматизації оцінювання навчальних робіт.

Розроблена система може бути впроваджена в університетах та освітніх платформах для скорочення часу перевірки робіт, підвищення об’єктивності оцінювання та надання студентам корисного зворотного зв’язку. У майбутньому можливе розширення функціоналу через інтеграцію з системами антиплагіату, адаптивною системою рекомендацій та аналітикою успішності на основі машинного навчання.

публікації

РОЗДІЛ 1. АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ

1.1 Постановка задачі

У сучасних умовах цифровізації освіти зростає потреба у створенні ефективних інструментів для автоматизації перевірки письмових робіт студентів. Традиційний процес оцінювання текстових завдань потребує значних витрат часу викладачів, оскільки включає аналіз структури тексту, змісту, орфографічних та граматичних помилок, а також загальної відповідності критеріям оцінювання.

Розробка веб-системи для автоматизованої перевірки студентських письмових робіт є актуальним завданням, що спрямоване на спрощення процесу аналізу текстових матеріалів, підвищення об'єктивності оцінювання та поліпшення зворотного зв’язку між викладачами та студентами.

Створений програмний комплекс дозволить:

- Автоматизувати перевірку письмових робіт на основі заданих критеріїв.
- Використовувати алгоритми обробки природної мови (NLP) для аналізу змісту.
- Формувати реферативний виклад роботи та надавати зворотний зв’язок користувачам.

Система буде реалізована у вигляді веб-додатку, який дозволятиме викладачам та студентам працювати в єдиному середовищі. Даний підхід спростить процес оцінювання та зробить навчальний процес більш ефективним.

Розроблювана система є програмним інструментом для автоматизованого аналізу студентських письмових робіт. Основною метою є полегшення перевірки документів для викладачів шляхом аналізу змісту, оцінювання певних критеріїв та автоматизованого виставлення оцінок.

Основні завдання, які система забезпечуватиме:

- Завантаження та обробку PDF-документів із текстовим вмістом.
- Аналіз письмових робіт за заданими критеріями (структура, зміст, оригінальність тощо).
- Автоматичне реферування (підсумковий виклад змісту).
- Використання методів обробки природної мови (NLP) для семантичного аналізу.
- Виставлення попередньої оцінки на основі аналізу.
- Надання можливості викладачу переглянути результати та скоригувати оцінку.

Функціональні вимоги представлені у вигляді таблиці 1.1, де кожна функція представлена з її описом відповідно.

Пілся аналогів

Таблиця 1.1

Функціональні вимоги

Функція	Опис
Інтерфейс користувача	Простий веб-додаток для завантаження файлів Відображення аналізу, підсумку та оцінки. Інтерактивні елементи для перегляду результатів та коригування оцінки.
Обробка документів	Видобування тексту з документів. Попередня перевірка якості тексту (розпізнавання нерозбірливих фрагментів).
Аналіз письмової роботи	Визначення ключових слів і основних тез документа. Перевірка структури (вступ, основна частина, висновки). Аналіз оригінальності за допомогою порівняння з базою попередніх робіт (векторна база даних). Визначення граматичних і стилістичних помилок.
Оцінювання	Розрахунок оцінки на основі аналізу. Можливість зміни оцінки викладачем.

Нефункціональні вимоги – це характеристики, які визначають якість роботи системи, її продуктивність, безпеку, масштабованість та інші параметри, що не залежать від конкретного функціоналу.

- Аналіз документа повинен тривати не більше 10 секунд.

- Доступ до системи повинен бути захищеним (автентифікація користувачів).
- Можливість обробки великої кількості документів одночасно.
- Підтримка останніх версій веб-браузерів.
- Документи та результати аналізу мають зберігатися у базі даних з можливістю подальшого доступу.

Із впровадженням даної системи навчальний процес стане більш ефективним, оскільки зменшить навантаження на викладачів, дозволить швидше надавати зворотний зв'язок студентам та забезпечить високу якість автоматичного оцінювання.

1.2 Аналіз існуючого програмного забезпечення за тематикою

Для ефективного вирішення задачі автоматизованого аналізу письмових робіт важливо розглянути існуючі рішення, які вже використовуються в освіті для перевірки текстових матеріалів. Нижче проаналізовано кілька категорій програмного забезпечення, що можуть бути корисними для розробки подібної системи.

Антиплагіатні сервіси

Системи для перевірки унікальності тексту, такі як Turnitin, Unicheck, Copyscape, Plagscan, дозволяють знаходити запозичення та плагіат у студентських роботах. Вони працюють на основі текстового аналізу та порівняння з базами наукових статей, книг і веб-сайтів.



Рис. 1.1. Логотипи антиплагіатних сервісів

Одним малюнком

Переваги:

- Висока точність виявлення збігів завдяки доступу до великих баз даних.
- Можливість інтеграції з освітніми платформами.
- Автоматичне генерування детальних звітів для викладачів.

Обмеження:

- Фокусуються лише на перевірці запозичень, не аналізуючи логіку викладення думок.
- Переважно є комерційними продуктами з обмеженим безкоштовним доступом.

Граматичні та стилістичні аналізатори

Системи такі як Grammarly, LanguageTool, ProWritingAid допомагають перевіряти граматику, пунктуацію та стилістичні помилки. Вони працюють на основі правил та штучного інтелекту.

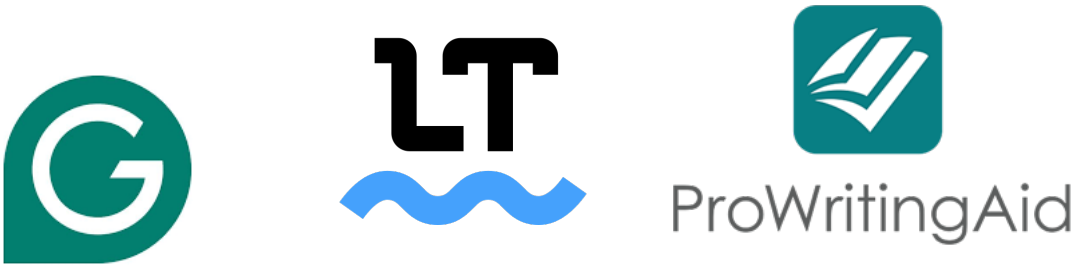


Рис. 1.2. Логотипи аналізаторів тексту

Переваги:

- Автоматичне виявлення граматичних помилок із поясненнями.
- Пропозиції щодо покращення стилю письма.
- Можливість інтеграції з текстовими редакторами та браузерами.

Обмеження:

- Аналізується тільки форма тексту, а не його зміст.
- Деякі сервіси не підтримують українську мову або мають обмежену функціональність у ній.

Засоби автоматизованого оцінювання текстів

Певні освітні платформи, наприклад EdX, Coursera, OpenAI API, використовують AI-моделі для оцінювання студентських робіт. Наприклад, GPT-4 або BERT можуть бути застосовані для аналізу логічності, послідовності викладення та відповідності темі.



Рис. 7-9. Логотипи засобів оцінювання текстів

Переваги:

- Можливість адаптації до різних предметів і тем.
- Використання штучного інтелекту для оцінки змісту роботи.
- Потенційно висока швидкість перевірки великої кількості робіт.

Обмеження:

- Висока вартість API для великих обсягів перевірок.
- Потреба в навчанні моделей для конкретних завдань.

На основі аналізу можна зробити такі висновки:

1. Антиплагіатні сервіси добре визначають запозичення, але не оцінюють змістовність роботи.

2. Граматичні аналізатори допомагають покращити якість тексту, але не аналізують його логіку та структуру.
3. Автоматизовані системи оцінювання частково вирішують проблему аналізу змісту, але мають високу вартість і вимагають адаптації.

Більшість інструментів або перевіряють плагіат, або граматику, але не поєднують їх із глибоким змістовним аналізом. Відсутня комплексна система, яка б враховувала логічну структуру, загальну якість роботи та відповідність заданим критеріям оцінювання. Запропонована система інтегруватиме аналіз змісту, оцінку логічності та коригування тексту, використовуючи сучасні AI-рішення на основі обробки природної мови (NLP). Це дозволить викладачам швидко й об’єктивно оцінювати якість письмових робіт, підвищуючи ефективність навчального процесу.

Таким чином, розробка нової системи є обґрунтованою, оскільки вона вирішуватиме комплексне завдання, яке не покривають наявні рішення.

Порівняльна характеристика. В табл
+ функціонал

1.3 Вибір архітектури

Для створення ефективної, продуктивної та надійної системи автоматизованого аналізу студентських письмових робіт застосовується комбінований підхід, що поєднує багаторівневу (N-tier) архітектуру та мікросервісний підхід. Таке поєднання забезпечує чіткий розподіл відповідальності між окремими компонентами, спрощує розробку, підтримку та подальше масштабування системи.

Основні принципи архітектури спрямовані на досягнення таких ключових вимог:

- Масштабованість – система повинна ефективно обробляти велику кількість робіт, не втрачаючи продуктивності.
- Гнучкість – можливість розширення та додавання нових функцій (порівняння текстів, антиплагіат, рекомендації).

- Доступність – система має працювати стабільно навіть при значному навантаженні.

Поєднання багаторівневої архітектури та мікросервісного підходу гарантує ефективний розподіл завдань між компонентами, спрощує управління системою, підвищує продуктивність і забезпечує її надійність та гнучкість для майбутніх покращень.

Багаторівнева (N-tier) архітектура

Багаторівнева архітектура передбачає розподіл системи на окремі рівні, кожен із яких виконує певні функції та відповідає за свою частину логіки. Такий підхід сприяє спрощенню розробки, підтримки та масштабування системи, оскільки зміни або оновлення в одному рівні не впливають на роботу інших компонентів.

У контексті нашої системи реалізовано чотири основні рівні, кожен з яких виконує конкретні завдання та забезпечує стабільну взаємодію між модулями. Це дає змогу покращити продуктивність, спростити відлагодження, а також забезпечити більш гнучку інтеграцію нових функціональних можливостей. Структура рівнів системи детально описана в таблиці 1.2.

Таблиця 1.2

Рівні архітектури

Рівень	Функціонал
Рівень представлення (Frontend/UI Layer)	Інтерфейс користувача (авторизація, завантаження робіт, перегляд аналізу).
Рівень бізнес-логіки (Business Logic Layer, BLL)	Обробка запитів від клієнтів та виконання основних операцій
Рівень доступу до даних (Data Access Layer, DAL)	Управління збереженням даних та робота з базою даних.
Рівень зберігання файлів (Storage Layer)	Зберігання та управління файлами студентських робіт.

Мікросервісна архітектура

Хоча система має чітку багаторівневу структуру, важливі функціональні компоненти будуть реалізовані як окремі мікросервіси. Це забезпечить додавання нових функцій без зміни всієї системи та незалежне розгортання та масштабування кожного сервісу. Перевага такого підходу – збій роботи одного сервісу не вплине на інші.

Основні сервіси системи:

- Сервіс авторизації та автентифікації (Auth Service) – управління користувачами та їх доступами.
- Сервіс аналізу текстів (Text Analysis Service) – обробка текстів для перевірки якості письмових робіт. Semantic Kernel використовуватиме векторні подання текстів для їх аналізу.
- Сервіс управління файлами (File Service) – збереження та обробка документів студентських робіт.
- Сервіс порівняння текстів (Plagiarism Detection Service) – перевірка на плагіат та пошук схожих текстів.

Запропонована архітектура поєднує N-tier структуру для розподілу логіки та мікросервісний підхід для масштабованості. Багаторівнева архітектура допомагає структуровано розділити логіку, та в свою чергу мікросервісна архітектура забезпечує гнучкість та надійність. Окрім того використання AI та NLP дозволяє ефективно аналізувати тексти.

1.4 Обґрунтування вибору інструментальних засобів та вимоги до апаратного забезпечення

Для реалізації системи автоматизованого аналізу студентських письмових робіт необхідно визначити оптимальний набір інструментів та технологій, які забезпечать ефективність, продуктивність, зручність розробки та подальшу підтримку.

Вибрані технології повинні безперешкодно взаємодіяти одна з одною, підтримуючи загальноприйняті протоколи та відкриті API, щоб інтеграція завершених модулів відбувалася органічно, без потреби в складних адаптерах або «костилях». При цьому система має гарантовано опрацьовувати великі масиви текстових даних із низькими затримками, адже від продуктивності безпосередньо залежить користувацький досвід і швидкість аналітичних операцій. Водночас архітектура повинна передбачати можливість поступового розширення її компонентів та ресурсів — без серйозних перебудов або замін ключових модулів, що дозволить адаптувати платформу під нові вимоги в довгостроковій перспективі. І нарешті, обрана стекова комбінація технологій мусить відповідати сучасним стандартам інформаційної безпеки, захищати конфіденційні дані викладачів і студентів та враховувати найновіші регуляторні вимоги й рекомендації щодо шифрування, автентифікації й аудиту доступу.

Backend: .NET Core + ASP.NET Web API

Для виконання поставленого завдання вирішено використовувати програмне забезпечення Microsoft, а саме ASP.NET Core. Даний інструмент надає високу продуктивність завдяки компіляції в машинний код, гнучкість у розробці API – підтримка RESTful API для інтеграції з іншими сервісами. Окрім зазначених переваг, .NET Core має вбудовану підтримка JWT, OAuth 2.0, ASP.NET Identity, для реалізації безпечного потоку даних на сервері. Однією з переваг .NET Core також є кросплатформеність – можливість розгортання на Windows, Linux, Docker для подальшого масштабування.

База даних: PostgreSQL + Vector Database

Відкритий код та стабільність – одна з найбільш надійних SQL-баз. Наведена БД має вбудовану підтримку JSONB – можливість зберігання напівструктурованих даних, та надає підтримку векторних індексів (PgVector) для NLP-завдань. Інтеграція з .NET через Entity Framework Core – зручна ORM для роботи з базою.

Frontend: React + TypeScript

Гнучкість та швидкість – динамічний інтерфейс без перезавантаження сторінки. Компонентний підхід – простота повторного використання компонентів. Строгість типізації – TypeScript допомагає уникати помилок при розробці продукту.

Аналіз тексту: Microsoft Semantic Kernel

Глибока інтеграція з Azure AI – доступ до потужних NLP-моделей, проте даний інструмент можна використовувати і з локально розгорнутими моделями, щоб уникнути непередбачених витрат. Semantic Kernel також надає можливість використання кастомних промптів та агентів. Оптимізація для .NET – проста інтеграція через SDK.

Вимоги до апаратного забезпечення

Наведена архітектура дозволяє розпаралелити процеси на частини: сервер (Backend API, NLP-модель, БД) та клієнт (Frontend). Цей підхід надає нам безмежні можливості для розгортання програмного забезпечення, локально – для розробки чи підтримки невеликої кількості користувачів, або ж з використанням хмарних сервісів. Нижче в табл. 1.3-1.4 наведено вимоги для апаратного забезпечення.

Таблиця 1.3

Серверні вимоги

Параметр	**Мінімальні вимоги**	**Рекомендовані вимоги**
Процесор	4 ядра (2.4 GHz)	8+ ядер (3.0 GHz)
ОЗП	8 GB	16+ GB
Сховище	50 GB SSD	200+ GB NVMe SSD
База даних	PostgreSQL 14+	PostgreSQL 15+
Мережа	100 Mbps	1 Gbps

Табл 1.4

Клієнтські вимоги (для користувачів)

Параметр	**Мінімальні вимоги**	**Рекомендовані вимоги**
Пристрій	ПК, ноутбук	ПК, ноутбук

Процесор	Intel i3 / AMD Ryzen 3	Intel i5+ / AMD Ryzen 5+
ОЗП	4 GB	8+ GB
Браузер	Chrome, Edge	Chrome, Edge

РОЗДІЛ 2. ПРОЕКТУВАННЯ СИСТЕМИ АВТОМАТИЗОВАНОГО АНАЛІЗУ СТУДЕНТСЬКИХ РОБІТ

2.1 Визначення варіантів використання та структури системи

У цьому підрозділі розглядаються ключові сценарії взаємодії користувачів із системою та її основні компоненти. Визначені варіанти використання лягають в основу UML-діаграми варіантів (Use Case) і відображають бізнес-процеси автоматизованого аналізу студентських робіт.

Таблиця 2.1

Актори системи

Актор	Роль у процесі
Студент	Завантажує PDF-роботу, переглядає власні подані завдання й результати аналізу та оцінювання.
Викладач	Переглядає автоматично згенерований аналіз, коригує оцінки та залишає текстові або числові коментарі.
Адміністратор	Налаштовує користувацькі ролі, курси, критерії оцінювання та моніторить статистику використання сервісу.

У межах кожного з цих акторів реалізовано низку типових варіантів використання. Кожен з цих сценаріїв моделюється на діаграмі Use Case (рис. 2.1) для подальшого формального опису послідовностей.

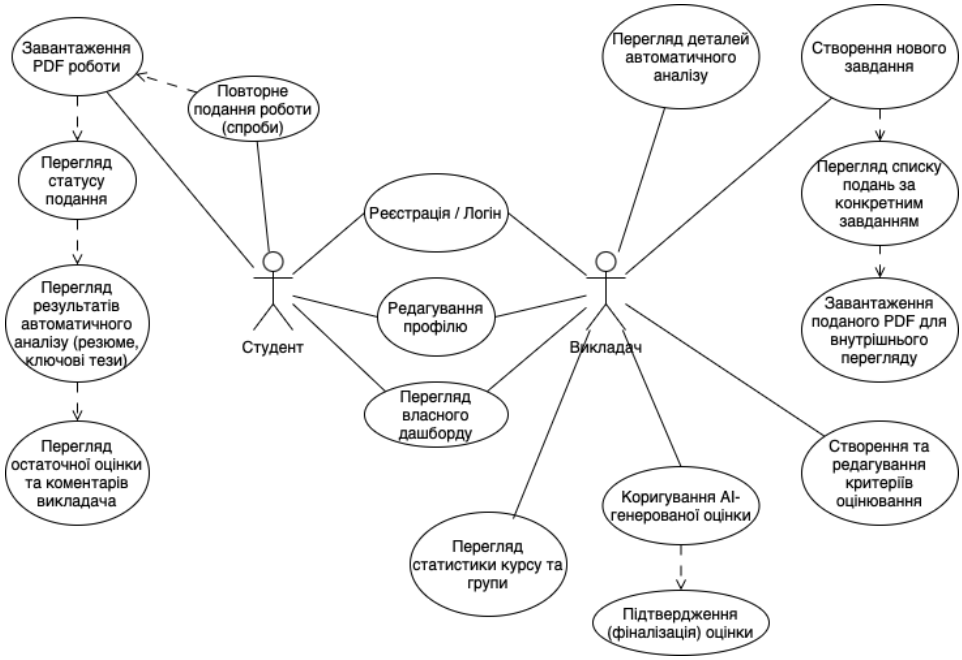


Рис. 2.1 Діаграма варіантів використання додатку

Кожен користувач системи починає роботу з етапу реєстрації або входу. Для цього необхідно ввести існуючі облікові дані або зареєструвати новий обліковий запис, вказавши логін та пароль чи електронну пошту. Після успішної валідації даних система генерує JSON Web Token (JWT), який дозволяє надалі автентифікуватися в усіх запитах до API. У разі коректного введення інформації користувач отримує повідомлення про вдало виконаний вхід, а токен зберігається в браузері для подальшої роботи.

Студент, після входу до системи, має можливість подати свою письмову роботу у форматі PDF. Для цього він обирає потрібний файл у файловій системі, після чого фронтенд надсилає його на сервер за допомогою захищеного запиту. Сервер приймає документ, перевіряє формат і розмір файлу, створює запис у таблиці Submission із вказанням ідентифікатора користувача, часу завантаження та шляху до файлу в Blob Storage. Після успішного завантаження студент бачить підтвердження прийому роботи на обробку.

Як тільки PDF-файл потрапляє на сервер, запускається послідовність модулів для обробки документу: спочатку виконується витяг тексту та його нормалізація, потім текст передається до Semantic Kernel для отримання векторних embedding-ів і побудови стислого резюме. Додатково запускаються алгоритми перевірки за критеріями: логічна структура, граматика та стиль. Результатом роботи AI-модуля стає об’єкт AnalysisResult, який зберігається в базі даних і містить згенероване резюме, набір ключових тез та попередній бал за заданими критеріями.

Після завершення автоматичного аналізу викладач може перейти до інтерфейсу перегляду результатів. Сторінка аналізу відображає короткий виклад основних ідей, список ключових тез та графічні індикатори відповідності роботи кожному з критеріїв. Завдяки чіткому і зрозумілому представленню даних викладач швидко оцінює якість робіт, не витрачаючи часу на ручний пошук ключових моментів у тексті.

Отримавши автоматично запропонований бал, викладач може змінити його відповідно до власного бачення та додати текстовий коментар. Інтерфейс дає можливість вказати остаточну оцінку, після чого зміни фіксуються у записі Grade: полях edited_grade та is_finalized. Після збереження викладач бачить оновлені дані в історії завдання, а студент отримує сповіщення про остаточний результат перевірки.

Рис. 2.2 Діаграма

2.2 Розробка бази даних системи

Для збереження метаданих студентських робіт, результатів їхнього аналізу та користувацьких профілів обрана реляційна СУБД PostgreSQL, що гарантує надійність, транзакційну цілісність та можливість масштабування. PDF-файли робіт зберігаються у шарі Blob Storage, у базі ж зберігаються лише шляхи до файлів і пов’язані з ними дані.

Таблиця	Короткий опис
User	Облікові записи користувачів: id, логін, email, повне ім’я, хеш пароля, посилання на роль
Student	Профіль студента: id, FK на Users, FK на StudentsGroup
Teacher	Профіль викладача: id, FK на Users
StudentsGroup	Групи студентів: id, назва
Course	Курси: id, заголовок, опис
Assignment	Завдання: id, FK на StudentsGroup, FK на Course, заголовок, опис, max_attempts, due_date
Document	Подання студентом роботи: id, FK на Users, FK на Assignment, номер спроби, шлях до файлу, час завантаження
Grade	Оцінювання: id, FK на Document, згенерована оцінка, змінена викладачем, статус фіналізації
GradingCriteria	Критерії оцінювання: id, FK на Users (творець), опис

	критерію, час створення
AnalysisResults	Результати аналізу: id, FK на Document, FK на GradingCriteria, FK на Grade, згенероване резюме, підтверджений

Для реалізації правильних зв’язків між сутностями використовувались всі три підходи для побудови взаємозв’язків у реляційних базах даних – один-до-одного, один-до-багатьох та багато-до-багатьох. Для правильної реалізації останнього було використано додаткові таблиці:

- TeacherGroup – зв’язок між викладачами та групами (teacher_id, group_id);
- CourseTeacher – між курсами та викладачами (course_id, teacher_id);
- GroupCourse – між групами і курсами (group_id, course_id).

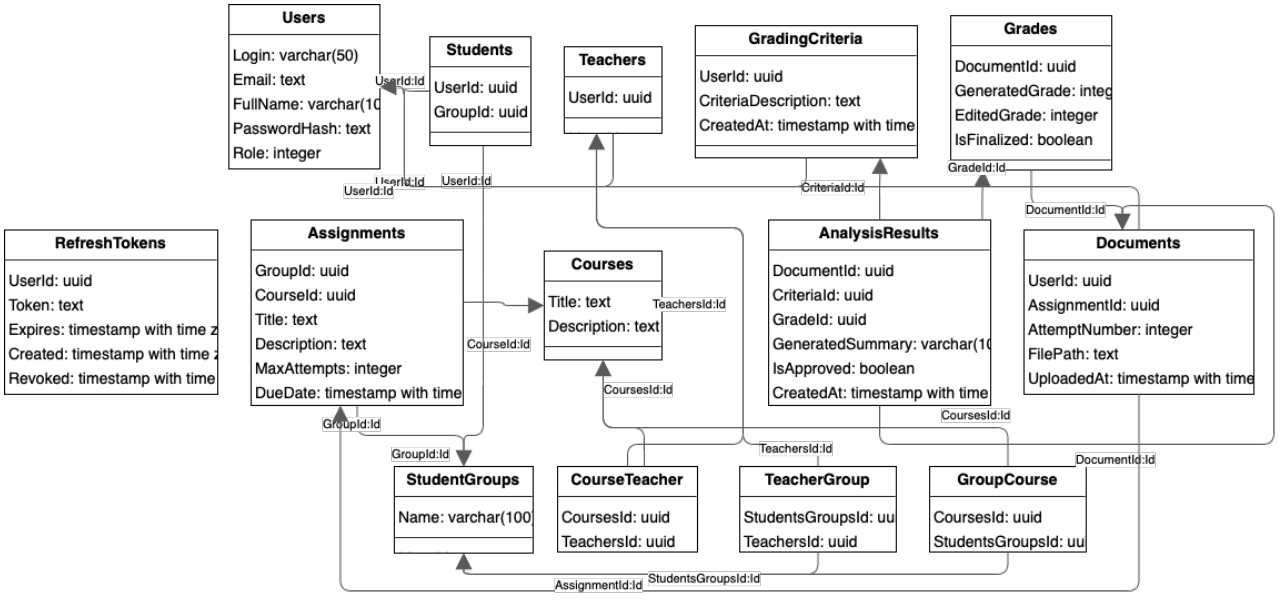


Рис. 2.3. Діаграма бази даних

Нижче наведено детальну інформацію про кожну із створених таблиць, їх поля, типи та призначення.

Таблиця Users зберігає базову інформацію про кожного зареєстрованого користувача. Для розрізнення профілів студентів і викладачів застосовуються таблиці Student і Teacher, у яких зберігаються додаткові поля (посилання на обліковий запис у Users та налаштування групи для студента). Таблиця StudentsGroup фіксує склади груп студентів, а Course – перелік навчальних курсів, доступних у системі.

Таблиця 2.3

Сутність User

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор користувача
login	varchar(100)	–	–	Логін для входу в систему
email	varchar(255)	–	–	Електронна адреса користувача
fullname	varchar(255)	–	–	Повне ім’я користувача
password_hash	varchar(255)	–	–	Хеш пароля
role_id	varchar(255)	–	Role.id	Хеш пароля

Таблиця 2.5

Сутність Student

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор студента
user_id	uuid	–	User.id	Посилання на обліковий запис користувача
group_id	uuid	–	StudentsGroup.id	Посилання на групу студента

Таблиця 2.6

Сутність Teacher

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор викладача
user_id	uuid	–	User.id	Посилання на обліковий запис викладача

Таблиця 2.6

Сутність StudentsGroup

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор
name	varchar(100)	–	–	Назва (шифр) групи студентів

Таблиця 2.6

Сутність Course

Поле	Тип	ПК	ФК	Призначення

id	uuid	+	–	Унікальний ідентифікатор курсу
title	varchar(100)	–	–	Заголовок курсу
description	text	–	–	Опис курсу

Окремо виділена таблиця RefreshTokens, яка керує механізмом поновлення сесій. Кожен запис містить ідентифікатор користувача (UserId), сам токен, часові позначки його створення, закінчення дії та відкликання. Ця модель гарантує безпечне зберігання й перевірку довгоживучих маркерів доступу.

Таблиця 2.6

Сутність RefreshTokens

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор
user_id	uuid	+	Users.id	Посилання на користувача
token	text	–	–	Згенерований JWT токен користувача
expires	timestamp	–	–	Час сгоряння токену
created	timestamp	–	–	Час створення токену
revoked	timestamp	–	–	Час оновлення токену

Сутності Assignment, Document, Grade, GradingCriteria та AnalysisResults відповідають за весь цикл роботи зі студентською роботою. Таблиця Assignment задає умови й дедлайни завдання, Document фіксує саме подання (PDF-файл і метадані завантаження). GradingCriteria містить описи правил оцінювання, а AnalysisResults зберігає результати AI-аналізу: згенероване резюме, відповідність критеріям і статус підтвердження. Таблиця Grade накопичує попередню оцінку від модуля й остаточну оцінку, відкориговану викладачем.

Таблиця 2.6

Сутність Assignment

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор завдання
group_id	uuid	–	StudentsGroup.id	Зв’язок із групою студентів
course_id	uuid	–	Course.id	Зв’язок із курсом

title	varchar(200)	–	–	Назва завдання
description	text	–	–	Детальний опис завдання
max_attempts	integer	–	–	Максимальна кількість спроб подання
due_date	timestamp	–	–	Кінцева дата подання роботи

Таблиця 2.6

Сутність Document

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор подання
user_id	uuid	–	Users.id	Посилання на студента, який подав роботу
assignment_id	uuid	–	Assignment.id	Посилання на відповідне завдання
attempt_number	integer	–	–	Номер спроби подання
file_path	varchar(255)	–	–	Шлях до PDF-файлу в Blob Storage
uploaded_at	timestamp	–	–	Дата і час завантаження

Таблиця 2.6

Сутність Grade

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор запису оцінювання
document_id	uuid	–	Document.id	Посилання на подання, яке оцінюється
generated_grade	numeric(5,2)	–	–	Оцінка, згенерована AI
edited_grade	numeric(5,2)	–	–	Оцінка, відкоригована викладачем
is_finalized	boolean	–	–	Позначка остаточного затвердження оцінки

Таблиця 2.6

Сутність GradingCriteria

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор користувача
login	varchar(100)	–	–	Логін для входу в систему
email	varchar(255)	–	–	Електронна адреса користувача
fullname	varchar(255)	–	–	Повне ім’я користувача
password_hash	varchar(255)	–	–	Хеш пароля
role_id	varchar(255)	–	(Role.id)	Хеш пароля

Таблиця 2.6

Сутність AnalisysResults

Поле	Тип	ПК	ФК	Призначення
id	uuid	+	–	Унікальний ідентифікатор результату аналізу
document_id	uuid	–	Document.id	Посилання на подання для аналізу
criteria_id	uuid	–	Grading Criteria.id	Посилання на критерій оцінювання
grade_id	uuid	–	Grade.id	Посилання на запис оцінки
generated_summary	text	–	–	Стислий виклад змісту роботи, згенерований AI
is_approved	boolean	–	–	Позначка, що результат аналізу підтверджено викладачем
created_at	timestamp	–	–	Дата та час створення запису результату аналізу

Для реалізації складних зв’язків (багато-до-багатьох) використовуються таблиці TeacherGroup, CourseTeacher та GroupCourse. TeacherGroup пов’язує викладачів із групами студентів, CourseTeacher — курси з викладачами, а GroupCourse — групи з курсами. Завдяки цим таблицям система може гнучко розподіляти права доступу та відображати навчальні плани, не дублюючи даних.

Таблиця 2.6

Сутність TeacherGroup

Поле	Тип	ПК	ФК	Призначення
teacher_id	uuid	+	Teacher.id	Посилання на викладача в групі
group_id	uuid	+	StudentsGroup.id	Посилання на групу студентів

Таблиця 2.6

Сутність CourseTeacher

Поле	Тип	ПК	ФК	Призначення
course_id	uuid	+	Course.id	Посилання на курс
teacher_id	uuid	+	Teacher.id	Посилання на викладача

Таблиця 2.6

Сутність GroupCourse

Поле	Тип	ПК	ФК	Призначення

group_id	uuid	+	StudentsGroup.id	Посилання на групу студентів
course_id	uuid	+	Course.id	Посилання на курс

2.3 Проектування та реалізація алгоритмів роботи системи

Алгоритмічний конвеєр обробки студентських робіт починається з отримання PDF-файлу як єдиного вхідного параметра та закінчується набором вихідних даних: стислим резюме, векторними embedding-ами документа, попереднім балом за критеріями та результатами семантичного пошуку схожих робіт. Спершу виконується витяг тексту з документа за допомогою PDF-парсера або OCR, що повертає один суцільний рядок тексту. Цей рядок проходить процедуру нормалізації, в ході якої видаляються зайві пробіли, розриви рядків і спеціальні символи, після чого текст розбивається на токени. Отриманий масив токенів передається у модуль Semantic Kernel, який генерує embedding-вектори як для всього документа, так і для його окремих сегментів, а також формує стислий виклад основних тез.

Рис. 2.4 Діаграма активності завантаження та обробки документів

На рівні побудови резюме алгоритм формулюється приблизно так: із кожного значущого параграфу вибираються ключові речення за допомогою кластеризації embedding-ів, зібрані речення упорядковуються за вихідним контекстом і з’єднуються в плавний короткий виклад. Після отримання резюме ScoreEngine застосовує множину критеріїв (структурність, логічність викладу, мовна коректність), кожен із яких вираховується як функція embedding-ів та статистичних ознак тексту, і підсумовує їх із заданими вагами, повертаючи початковий бал.

Рис. 2.5 Діаграма активності для створення резюме

Оцінка складності основних кроків показує, що витяг і нормалізація тексту працюють у лінійному часі від обсягу символів O(n), а побудова embedding-ів та резюмування залежить від внутрішньої складності LLM-модуля (зазвичай O(n·d) для n–токенів та d–розмірності векторів). Запит до векторної бази даних для пошуку схожих документів має складність O(log N) при наявності індексів pgvector.

У майбутньому алгоритмічну лінію можна доповнити підтримкою нових мов через додаткові OCR-модулі, альтернативними стратегіями резюмування (наприклад, TextRank) або додати критерії, які аналізують цитування та стиль оформлення. Ці точки розширення закладені в архітектурному дизайні як окремі інтерфейси сервісів, що дозволяє впроваджувати нові компоненти без зміни існуючого конвеєра.

2.4 Реалізація серверної частини системи

Серверну частину “Student Grading System” реалізовано з принципом багаторівневої (N-tier) архітектури, що дозволяє чітко відокремити зони відповідальності й полегшити підтримку та тестування кожного рівня. Угорі знаходяться контролери ASP.NET Core Web API, які приймають HTTP-запити, проводять базову валідацію через атрибути та передають обробку далі. У шарі бізнес-логіки (BLL) згруповано всі сервіси: AuthService для реєстрації, входу й оновлення токенів, AssignmentService для CRUD-операцій із завданнями, DocumentService для завантаження PDF-файлів і FileStorageService, який працює із Blob Storage, а також AnalysisService, що інкапсулює виклики Semantic Kernel і формування попереднього результату аналізу.

Доступ до бази даних виконано через Entity Framework Core з реалізацією патерну Repository: кожна сутність має окремий репозиторій, що інкапсулює стандартні операції Create/Read/Update/Delete, а також спеціалізовані методи для складних запитів (наприклад, пошук по векторним embedding-ам через pgvector). Це дозволяє відокремити написання LINQ-виразів від бізнес-логіки та легко замінити сховище даних без зміни шарів вище.

Укладення та тестування контрактів API здійснено за допомогою OpenAPI (Swagger) і окремо задокументовано в Postman-колекції, яка детальніше описана нижче. У ній зібрані всі запити до кінцевих точок: від реєстрації й аутентифікації до завантаження документів, запуску аналізу та коригування оцінок.

Далі наведено розподіл усіх кінцевих точок (ендпоінтів) за функціональними групами. Кожна група відповідає за певний набір операцій у системі.

Таблиця 2.6

Групи API-ендпоінтів

Методи	Префікс маршруту	Опис групи
POST	/api/auth	Реєстрація користувача, вхід у систему та оновлення JWT-токена через refresh-токен
GET	/api/users	Отримання даних поточного користувача, списків викладачів, студентів та учасників груп
POST, GET	/api/courses	Створення нових курсів; отримання інформації про курс, курси викладача або курси студента
POST, GET	/api/assignments	Створення завдань для групи; отримання деталей конкретного завдання та списку завдань груп
POST, GET	/api/criteria	Створення критеріїв оцінювання; запит інформації про один критерій або всі критерії
POST, GET	/api/document	Завантаження PDF-файлів у систему; отримання метаданих одного подання або всіх подань користувача
POST, GET, PUT	/api/grading	Запуск автоматичного аналізу документу; запит результатів аналізу; редагування та фіналізація оцінок

Рис. 2.6 Діаграма компонентів сервера

Завдяки такій організації серверної частини можна гнучко додавати нові сервіси, масштабувати окремі компоненти та забезпечувати стабільну роботу під високим навантаженням.

2.5 Реалізація автоматизованого аналізу письмових робіт за допомогою AI

2.6 Реалізація веб-інтерфейсу системи

РОЗДІЛ 3.

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Component Diagram Tutorial [Електронний ресурс] – Режим доступу до ресурсу: https://www.lucidchart.com/pages/uml-component-diagram.

2. Karcic F. A Developer's Guide to CQRS Using .NET Core and MediatR [Електронний ресурс] / Faris Karcic. – 2020. – Режим доступу до ресурсу: https://www.rubicon-world.com/blog/2020/06/a-developers-guide-to-cqrs-using-net-core-and-mediatr/.

3. Understanding NLP Architecture [Електронний ресурс] – Режим доступу до ресурсу: https://medium.com/@bipasha_bg/understanding-nlp-architecture-3b0a50c8192f.

4. ML.NET – An open source and cross-platform machine learning framework [Електронний ресурс] – Режим доступу до ресурсу: https://dotnet.microsoft.com/en-us/apps/ai/ml-dotnet.

5. Про патерни проектування [Електронний ресурс] – Режим доступу до ресурсу: https://foxminded.ua/paterny-proektuvannia/.

6. Dependecy injection та Singletone. Все, що треба знати, і ще трошки [Електронний ресурс] – Режим доступу до ресурсу: https://medium.com/@ivanfomenko/dependecy-injection-та-singletone-все-що-треба-знати-і-ще-трошки-2e738373eb94.

7. Large Language Models [Електронний ресурс] – Режим доступу до ресурсу: https://www.udemy.com/course/large-language-models-llms-fundamentals/

8. N-Layer Architecture In A Multi-Project Solution [Електронний ресурс] – Режим доступу до ресурсу: https://makeitnew.io/n-layer-architecture-in-a-multi-project-solution-d72604f0e030.

9. OpenAPI document generation in .NET 9 [Електронний ресурс] – Режим доступу до ресурсу: https://devblogs.microsoft.com/dotnet/dotnet9-openapi/

10. DeepSeek AI [Електронний ресурс] – Режим доступу до ресурсу: https://aichatbot.co/onboarding-deepseek

11.